IFT 211: Digital Logic Design

Lecture 1: Introduction to Information Representation and Number Systems

1. Introduction

Digital systems—including computers, smartphones, and embedded devices—process a wide variety of information such as numbers, texts, images, audio, and video. Despite this diversity, all such information is internally represented and manipulated in **binary form**, i.e., as sequences of **0s and 1s**.

Why Do Computers Use the Binary Number System?

Computers and other digital devices rely on the binary number system for several important reasons:

1. **Compatibility with Electronic Components**
   Electronic components inherently operate using two distinct states. For example, a switch can be either *on* or *off*, and a transistor can be either *conducting (1)* or *non-conducting (0)*. This two-state behavior aligns naturally with the binary system, making it an ideal choice for digital circuit design.

2. **Simplicity of Operations**
   All arithmetic and logical operations performed in the decimal system can also be carried out in the binary system. This ensures that binary representation does not limit computational capability.

3. **Efficiency in Design and Reliability**
   By using only two digits (0 and 1), digital systems become simpler to design compared to systems that would require handling ten distinct decimal digits. This simplicity reduces hardware complexity, lowers production costs, and enhances the overall reliability of the system

1.1 Data Representation in Digital Systems

Data refers to symbols that represent entities such as people, objects, events, or ideas.

Examples include: Names and addresses, Numerical values, Images and colors, and Audio signals.

Data representation is the method by which such information is encoded into a form suitable for storage, processing, and transmission in digital systems.

Digital devices store all forms of data as **binary sequences**, enabling efficient processing by electronic circuitry.

Units of Data

The basic unit of measure for data in modern computers and other digital devices is the bit, which can take the values 0 or 1. Each group of 8 bits constitutes 1 Byte, symbolized by the character B.

The number of distinct states representable with n bits is 2^n

Examples

• With 3 bits, we can represent $2^3 = 8$ different states.

• With 4 bits, we can represent $2^4 = 16$ different states.

Common Data Size Units

Unit	Description
Kilobyte (KB)	(10^3) bytes
Megabyte (MB)	(10^6) bytes
Gigabyte (GB)	(10^9) bytes
Terabyte (TB)	(10^{12}) bytes

1.2 Data Representation using Computer Codes

Digital devices use a binary coding scheme known as **Computer Codes** for internal data representation and manipulation. Since digital systems operate using only two states (0 and 1), all forms of data must be converted into binary codes before they can be stored, processed, or transmitted. Computer codes are therefore used not only in computers but also in calculators, mobile phones, digital clocks, ATMs, smart TVs, microcontrollers, and other electronic devices. Some of the commonly used computer codes include:

1.2.1 Binary Coded Decimal (BCD) Code

The BCD Code is also known as the **Binary Coded Decimal** code. It is one of the early memory codes. In this coding method, each digit of a Decimal number is converted into its Binary equivalent. In this method, the entire Decimal number is not converted into a pure Binary form, but only the 10 decimal digits. The BCD equivalent of Decimal digits from 0 to 9 is shown in the table below.

Decimal Digits	BCD Code
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

BCD equivalent of Decimal Digits from 0 to 9

4 bits are used to represent each digit in this coding system because digits 8 and 9 requires 4 bits.

Let's take an example, the pure Binary form of Decimal number 45_{10} is 101101_2. But if we convert 45_{10} into BCD then it will be 01000101. Here 0100 is written for 4 and 0101 is written for 5. Here we need to note that each Decimal digit is independently converted into a 4 bit Binary number.

This method of conversion is easier. One more thing we need to note that when 4 bits are used, 16 configurations are possible. But only first 10 combinations are used. The remaining 6 combinations 1010, 1011, 1100, 1101 and 1111 are not used in BCD coding. Because these combinations have Decimal values from 10 to 15. Thus from 1010 to 1111 do not represent 10 to 15 in BCD. Hence equivalent codes of 10 to 15 will be as.

10_{10} = 00010000 in BCD

11_{10} = 00010001 in BCD

12_{10} = 00010010 in BCD

13_{10} = 00010011 in BCD

14_{10} = 00010100 in BCD

15_{10} = 00010101 in BCD

With a 4-bit BCD coding system, we can only represent Decimal numbers. A 4-bit BCD cannot be used to represent various characters used by a computer. Thus, instead of using 4-bit BCD, computer designers use 6-bit BCD to represent various characters. Because with 4 bits there will be only 16 combinations, and for 6 bits there will be 64 possible combinations. 6-bit BCD is sufficient to code the Decimal digits, alphabetical digits, and special characters. 6-bit BCD code is obtained by adding two additional zone positions to the 4-bit BCD code.

6-bit BCD codes of numeric and alphabetic data are shown in the table below.

Character	BCD Code		Character	BCD Code	
	Zone	Digit		Zone	Digit
A	**11**	**0001**	**S**	**01**	**0010**
B	11	0010	T	01	0011
C	11	0011	U	01	0100
D	11	0100	V	01	0101
E	11	0101	W	01	0110
F	11	0110	X	01	0111
G	11	0111	Y	01	1000
H	11	1000	Z	01	1001
I	11	1001	**0**	**00**	**0000**
J	**10**	**0001**	1	00	0001
K	10	0010	2	00	0010
L	10	0011	3	00	0011
M	10	0100	4	00	0100
N	10	0101	5	00	0101
O	10	0110	6	00	0110
P	10	0111	7	00	0111
Q	10	1000	8	00	1000
R	10	1001	9	00	1001

BCD CODE OF ALPHABETIC AND NUMERIC CHARACTERS

Example: Find the BCD Code of the word TRAIN in Binary notation?

From the above table, we will write code for each character of the word TRAIN, then we will combine them all.

BCD notation of T = 010011
BCD notation of R = 101001

BCD notation of A = 110001

BCD notation of I = 111001

BCD notation of N = 100101

Thus, the BCD code for the word TRAIN is 010011101001110001111001100101.

Exercise

Find the BCD code of the following words and hence determine the number of bits needed to represent each.

i.

UNIVERSITY

ii. GOMBE iii.

COMPUTER

1.2.2 Extended Binary Coded Decimal Interchange Code (EBCDIC)

The main problem with BCD codes is that they can only represent 64 different characters. Thus, they are insufficient to represent Decimal Numbers (10), Lower Case Alphabetic letters (26), Upper Case Alphabetic letters (26), and Special Characters (more than 28). Hence, 6-bit BCD codes are extended to 8-bit BCD codes. The extra 2 bits are added to the Zone bits. Now the Zone is extended to 4 bits. After extension to Zone bits, the resulting code is known as **Extended Binary Coded Decimal Interchange Code** or **EBCDIC**. With this new coding method, it is possible to represent 256 different characters in a computer memory. A large variety of printable characters and many non-printable control characters can be represented. Control characters are used to control the printer's vertical spacing, movement of the cursor on the screen, etc. Since EBCDIC is an 8-bit code, it is divided into groups of 4 bits. The first 4 bits from the left side represent Zone Codes. EBCDIC is developed by IBM. Numbers coded in EBCDIC are called Zoned Decimal Numbers. Numeric data input into a computer is usually represented by decimal numbers. The table for EBCDIC codes of numeric and alphabetic characters is shown below.

Characters	EBCDIC		Characters	EBCDIC		Characters	EBCDIC	
	Zone	Digit		Zone	Digit		Zone	Digit
A	**1100**	**0001**	**S**	**1110**	**0010**	**j**	**1001**	**0001**
B	1100	0010	T	1110	0011	k	1001	0010
C	1100	0011	U	1110	0100	l	1001	0011
D	1100	0100	V	1110	0101	m	1001	0100
E	1100	0101	W	1110	0110	n	1001	0101
F	1100	0110	X	1110	0111	o	1001	0110
G	1100	0111	Y	1110	1000	p	1001	0111
H	1100	1000	Z	1110	1001	q	1001	1000
I	1100	1001	**a**	**1000**	**0001**	**r**	**1001**	**1001**
J	**1101**	**0001**	b	1000	0010	s	1010	0010
K	1101	0010	c	1000	0011	t	1010	0011
L	1101	0011	d	1000	0100	u	1010	0100
M	1101	0100	e	1000	0101	v	1010	0101
N	1101	0101	f	1000	0110	w	1010	0110
O	1101	0110	g	1000	0111	x	1010	0111
P	1101	0111	h	1000	1000	y	1010	1000
Q	1101	1000	i	1000	1001	z	1010	1001
R	1101	1001						

EBCDIC CODE FOR LOWER AND UPPER CASE ALPHABETIC LETTERS

Example: Using Binary notation write EBCDIC for the word BASE?

EBCDIC code of word BASE can be written from the above EBCDIC Table of Alphabets.

EBCDIC notation for character B = 11000010
EBCDIC notation for character A = 11000001
EBCDIC notation for character S = 11100010
EBCDIC notation for character E = 11000101

Therefore EBCDIC code for BASE is 11000010110000011110001011000101.

Numeric Digit	EBCDIC Code	
	Zone	Digit
0	1111	0000
1	1111	0001
2	1111	0010
3	1111	0011
4	1111	0100
5	1111	0101
6	1111	0110
7	1111	0111
8	1111	1000
9	1111	1001

EBCDIC CODE FOR NUMERIC DIGITS

Example: Write the number 345 in EBCDIC coding using binary notation?

EBCDIC code for digit 3 = 11110011
EBCDIC code for digit 4 = 11110100
EBCDIC code for digit 5 = 11110101

Thus, EBCDIC code for number 345 is 111100111111010011110101.

Exercise

1. Using Binary notation, write the EBCDIC code for the following words
   i. STUDENT
   ii. LECTURER
2. Write the following numbers in EBCDIC coding using binary notation
   i. 3848 ii. 2348 iii. 1234

1.2.3 American Standard Code for Information Interchange (ASCII) Codes

ASCII, pronounced as "Askey" stands for **American Standard Code for Information Interchange**. It is a widely known character coding scheme used for the internal representation of data in computers and other digital devices. ASCII is of two types: ASCII-7 and ASCII-8. ASCII7 is a 7-bit code; thus, it can represent only 128 different characters. In

this method of coding first 3 bits are used as Zone bits and the last 4 bits represent Digit and character. A table for Numeric and Alphabetic character in ASCII-7 is shown below.

Character	ASCII-7 Code		Character	ASCII-7 Code	
	Zone	Digit		Zone	Digit
A	**100**	**0001**	S	101	0011
B	100	0010	T	101	0100
C	100	0011	U	101	0101
D	100	0100	V	101	0110
E	100	0101	W	101	0111
F	100	0110	X	101	1000
G	100	0111	Y	101	1001
H	100	1000	Z	101	1010
I	100	1001	**0**	**011**	**0000**
J	100	1010	1	011	0001
K	100	1011	2	011	0010
L	100	1100	3	011	0011
M	100	1101	4	011	0100
N	100	1110	5	011	0101
O	100	1111	6	011	0110
P	**101**	**0000**	7	011	0111
Q	101	0001	8	011	1000
R	101	0010	9	011	1001

Similarly, ASCII-8 is an extended version of ASCII-7. It is an 8-bit code. Thus, it can represent 256 different characters. One additional bit is added to the Zone bit. Now the Zone bits are 4. A table for Numeric and Alphabetic characters in ASCII-8 is shown below. The Zone bits of ASCII-7 and ASCII8 are different, but the digits are identical.

Character	ASCII-8 Code		Character	ASCII-7 Code	
	Zone	Digit		Zone	Digit
A	**1010**	**0001**	S	1011	0011
B	1010	0010	T	1011	0100
C	1010	0011	U	1011	0101
D	1010	0100	V	1011	0110
E	1010	0101	W	1011	0111
F	1010	0110	X	1011	1000
G	1010	0111	Y	1011	1001
H	1010	1000	Z	1011	1010
I	1010	1001	**0**	**0101**	**0000**
J	1010	1010	1	0101	0001
K	1010	1011	2	0101	0010
L	1010	1100	3	0101	0011
M	1010	1101	4	0101	0100
N	1010	1110	5	0101	0101
O	1010	1111	6	0101	0110
P	**1011**	**0000**	7	0101	0111
Q	1011	0001	8	0101	1000
R	1011	0010	9	0101	1001

Example: Write the binary coding for the word GATE in ASCII-7 and in ASCII-8?

From ASCII-7 table we can write the binary codes for each character as follows

ASCII-7 code for character G = 1000111

ASCII-7 code for character A = 1000001

ASCII-7 code for character T = 1010100

ASCII-7 code for character E = 1000101

Thus, the binary notation for the word GATE in ASCII-7 is 1000111100000110101001000101

From ASCII-8 table we can write the binary codes for each character as follows

A SCII-8 code for character G = 10100111

ASCII-8 code for character A =10100001
ASCII-8 code for character T = 10110100

ASCII-8 code for character E = 10100101

Thus, the binary notation for the word GATE in ASCII-8 is

10100111101000011011010010100101.

Exercise

1. Write the binary coding for the following words in ASCII-8 and hence determine the number of bytes needed for the representation of each word:

i. KEYBO ARD
ii. MOUSE
iii. LEARNING
iv. NETWORK

1.2. 4 Unicode

Unicode is a worldwide character-encoding standard developed to accommodate more characters and special symbols and to support all the languages of the world. Unlike ASCII, which uses 7 bits or 8 bits for each character, Unicode uses 16 bits, which means that it can represent more than 65,000 unique characters.

The basic idea of Unicode is to have a single encoding for the whole world, an encoding that can support the characters of all the languages in the world.

Unicode provides a unique code for every character, no matter what platform, no matter what program, and no matter what language.

2. NUMBER SYSTEMS

A number system is a way of representing numbers by an ordered set of symbols, called digits, with relations defined for mathematical operations such as addition, subtraction, and multiplication. It can also be defined as a set of values used to represent different quantities.

For example, a number system can be used to represent:

i. The number of staff in a department

ii. The score obtained by a student in his/her test or examination

iii. the population of a state or a country

iv. data and information in a computer

v. the amount of money one has in their bank account

There are two types of number systems: positional and non-positional systems.

2.1 NON-POSITIONAL NUMBER SYSTEM

A non-positional number system is a system in which the position of the digits is not significant to the value of the number. Or to say it in another way, the value of each digit in a non-positional number system is a constant and does not strictly depend on its position in the number.

Examples of the non-positional number system are the Roman and Greek number system. All non-positional number system has a common drawback- the notation of big numbers in them is inefficient. Consequently, they have gained only limited use as this could lead to inaccuracy when determining the values of the numbers.

In summary, the characteristics of the non-positional number system can be broken down as follows:

- Each symbol represents the same value regardless of its position in the number
- Use symbols such as I for 1, II for 2, III for 3, etc.

- The symbols are usually added or subtracted depending on the arrangement of the symbols to find out the value of a particular number.
- It is difficult to perform arithmetic with such a number

2.2 POSITIONAL NUMBER SYSTEMS

A positional number system is a system in which the position of the digits is significant for the value of the number. This means that the value of the digits in the number is not strictly defined and depends on which position the digit is. Or to say it in another way, the positional number system gives different meanings to the same digit depending on its position in the number. For example, in the number 351, the digit 1 has a value of 1, whereas in the number 1024, it has the value of 1000.

The radix (r) or base of a number system is the total number of digits allowed in the number system. Positional number systems commonly used in digital systems and computer programming include:

Decimal (r=10)

Binary (r =2)

Octal (r = 8)

Hexadecimal (r=16)

In summary, the characteristics of positional number system can be broken down as follows:

- Use only few symbols call digits
- These symbols represent different values depending on the position they occupy in the number
- The value of each digit is determined by:
    i. The digit itself
    ii. The position of the digit in the number

iii. The base of the number system

- The maximum value of a single digit is always one less than the value of the base

In the next section, we are going to look at the different types of positional number systems. However, before we jump into the various types of positional number systems, it is worth mentioning that all positional number systems have the same logical settings, the main difference between one positional number system and another lie in their number of bases, i.e., the number of digits each support.

2.2.1 EXAMPLES OF POSITIONAL NUMBER SYSTEMS

i. Decimal Number System

The decimal number system is a positional number system that has ten (10) symbols or digits (0,1,2,3,4,5,6,7,8,9). Hence, the decimal number system has a base of 10. The maximum value of a single digit in this number system is 9 (one less than the base). Each position of a digit represents a specific power of 10. The decimal number system is the most widely spread. Perhaps this is because it enables us to count on our hands.

Examples

$2541_{10} = (2*10^3) + (5*10^2) + (4*10^1) + (1*10^0)$

$= 2000 + 500 + 40 + 1$

$= 2541$

$1254_{10} = (1*10^3) + (2*10^2) + (5*10^1) + (4*10^0)$

$= 1000 + 200 + 50 + 4$

$= 1254$

ii. Binary Number System

The binary number system is a positional number system that has only two symbols or digits (0 and 1). Hence, its base is 2. The maximum value for a single digit in this numbering system is 1 (one less than the value of the base). Similarly, just like the decimal number system, each position of a digit in the binary number system represents a specific power of the base (2). The binary number system is the number system used in computers and other digital devices.

Examples

$10101_2 = (1*2^4) + (0*2^3) + (1*2^2) + (0*2^1) + (1*2^0)$

$= 16 + 0 + 4 + 0 + 1$

$= 21_{10}$

Check to see whether $21_{10} = 10101_2$

iii. Octal Number System

A positional number system that has 8 symbols or digits (0, 1, 2, 3, 4, 5, 6, 7). Hence, it has a base of 8. The maximum value for a single digit is 7 (one less than the value of the base). Each position of a digit represents a specific power of the base (8). Since there are only eight digits, 3 bits ($2^3 = 8$) are sufficient to represent any octal number in binary.

For example,

$2057_8 = (2*8^3) + (0*8^2) + (5*8^1) + (7*8^0)$

$= 1024 + 0 + 40 + 7$

$=1071_{10}$

iv. Hexadecimal number System

A positional number system that has 16 symbols or digits (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E,

F). Hence, it has a base of 16. The symbols A, B, C, D, E, and F represent the decimal values 10, 11, 12, 13, 14, and 15 respectively

The maximum value for a single digit is 15 (one less than the value of the base). Each position of a digit represent a specific power of the base (16). Since there are only sixteen digits, 4 bits ($2^4 = 16$) are sufficient to represent any hexadecimal number in binary.

Example

$1FB_{16} = (1*16^2) + (F * 16^1) + (B*16^0)$

$= 1*256 + 15 *16 + 11 * 1$

$= 256 + 240 + 11$

$= 507_{10}$

2.3 INTER –CONVERSIONS

In this section, we will learn how to convert numbers from one number system to another number system.

i. Converting a number from another base to a decimal number (base 10)

Example

Convert the following numbers to decimal

1. $1EAF_{16}$
2. 10100_2

Solutions

1. $1EAF_{16} = (1 *16^3) + (E*16^2) + (A*16^1) + (F*16^0)$

$= 1 *4096 + 14*256 + 10*16 + 15 *1$

$= 4096 + 3584 + 160 +15$

$= 7855$

$1EAF_{16} = 7855_{10}$

2. $10100_2 = (1*2^4) + (0*2^3) + (1*2^2) + (0*2^1) + (0*2^0)$

$= 1*16 + 0*8 + 1*4 + 0*2 + 0*1$

$= 16 + 4$

$= 20$

Therefore, $10100_2 = 20_{10}$

Exercise

Convert the following number to decimal

1. $7FA_{16}$, $A5_{16}$, $BEFA_{16}$ $(2C6B.F2)_{16}$, $(306.D)_{16}$
2. $(673.124)_8$
3. 1011011_2 , 01010101_2 , 00000111_2

ii. Conversion of decimal number to binary number

Class work, Convert the following decimal numbers to binary

1. 34_{10}
2. 45_{10}

Exercise

1. Convert 30_{10} to binary
2. Convert 55_{10} to binary
3. Convert 20_{10} to binary